

FPGA-Based Smart Hospital Emergency Response Dispatcher: VHDL Design, Verification, FPGA Demonstration, and Altium PCB Realization

Hardware Engineering Laboratory – Team C2

Bertrand Mungu Cho, Daniel Chidi Chimezie, Riway Ghimire, Stephen Uzochi Obuh

Hochschule Hamm-Lippstadt, Germany

Summer Semester 2026

Abstract—This paper documents the development and successful laboratory demonstration of an FPGA-based Smart Hospital Emergency Response Dispatcher. The system was implemented in VHDL for the Digilent Nexys A7 platform and integrates patient registration, triage priority encoding, room/doctor/equipment allocation, patient parking, selective release, emergency override, fault supervision, LED indication, seven-segment visualization, and UART status transmission. The repository contains ten synthesizable VHDL modules, ten dedicated testbenches, and 114 assertion statements. The FPGA implementation was complemented by a seven-sheet Altium schematic design, a two-layer printed circuit board (PCB), a three-dimensional PCB model, a bill of materials, and complete Gerber and drill outputs. The final laboratory demonstration confirmed the intended operating scenarios. Each design and validation figure is presented individually to preserve readability and provide traceable evidence of the team’s work.

Index Terms—FPGA, VHDL, Nexys A7, Artix-7, finite-state machine, hospital dispatcher, Altium Designer, PCB, hardware verification.

I. INTRODUCTION

Field-programmable gate arrays (FPGAs) provide deterministic parallel processing, direct hardware access, and precise state-machine control. These characteristics make them suitable for teaching digital design and for prototyping resource-management controllers. Team C2 developed a Smart Hospital Emergency Response Dispatcher that models the reception, triage, allocation, treatment, parking, release, and fault-handling stages of an emergency department.

The target platform was the Digilent Nexys A7, based on an AMD Artix-7 FPGA and equipped with a 100-MHz clock, switches, push buttons, LEDs, seven-segment displays, and communication interfaces [1], [2]. The RTL was written in accordance with IEEE VHDL and multivalued logic standards [3], [4]. Vivado Design Suite was used for simulation, synthesis, constraints, implementation, and bitstream generation following the vendor design flow [5]–[8].

The work demonstrates an end-to-end hardware engineering process: modular RTL development, assertion-based verification, FPGA programming, interactive laboratory validation, schematic capture, PCB layout, three-dimensional inspection,

and manufacture-oriented output generation. Detailed post-implementation Vivado records are intentionally excluded from this report; the focus is the verified design architecture, functional scenarios, PCB realization, and the team’s final hardware demonstration.

II. DEVELOPMENT PATH AND SYSTEM ARCHITECTURE

The laboratory activities progressed from elementary combinational logic through arithmetic circuits, clocked counters, FPGA constraints, and PCB design before integration into the final project. This staged approach reduced integration risk and established reusable design skills [9], [10].

The final system is structurally divided into ten synthesizable modules: patient acquisition, priority encoding, resource management, scheduler finite-state machine (FSM), patient table, safety supervisor, LED controller, seven-segment driver, UART controller, and the top-level integration entity. The architecture captures patient data, computes a priority, tests resource availability, allocates room/doctor/equipment indices, and exposes the current state through board outputs.

Figure 1 presents the dispatcher state progression. The sequence begins in `IDLE`, accepts a patient, performs triage, checks available resources, allocates the required resources, dispatches treatment, monitors the patient, releases resources, and returns to the initial state. Explicit fault and emergency paths provide defined behavior when normal scheduling cannot continue.

Figure 2 shows the same operating cycle as a presentation-oriented state sequence used during the demonstration. It makes the progression from reception to release visible to observers and supports the explanation of the controller’s temporal behavior.

III. RTL DESIGN AND VERIFICATION

The `patient_generator` synchronizes external inputs and captures a new transaction on a detected control edge. The `priority_encoder` maps patient type and severity to scheduling information. The `resource_manager` maintains occupancy vectors for four rooms, four doctors, and four

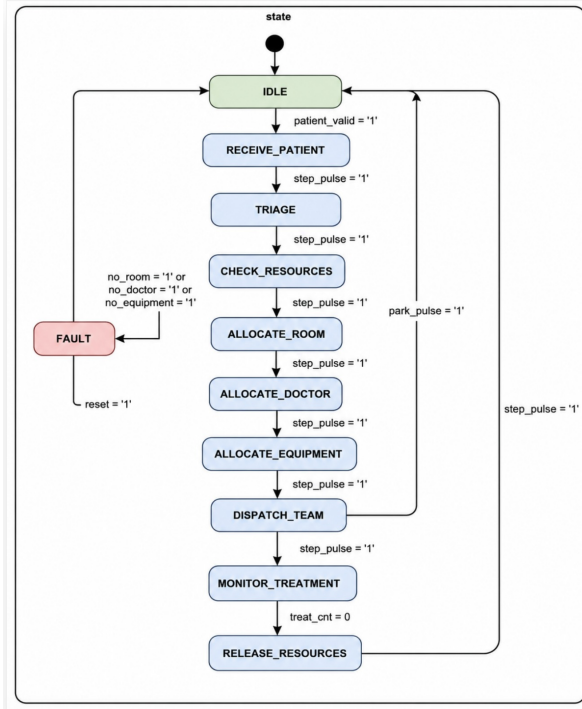


Fig. 1. Dispatcher finite-state-machine workflow used to structure the complete patient-handling cycle.

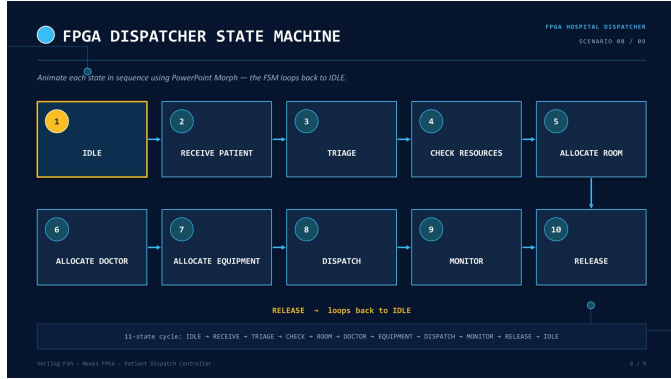


Fig. 2. Demonstration view of the sequential dispatcher states from patient reception to resource release.

equipment units. The `scheduler_fsm` sequences all control actions, while the `patient_table` stores up to four parked assignments for later selective release.

The `safety_supervisor` latches abnormal conditions, the `led_controller` maps occupancy and alarms to the 16 board LEDs, and the `seven_segment_driver` multiplexes status information. The `uart_controller` provides an 8N1 serial output at 115200 baud, and `top_level` integrates the design and board interface.

Verification was performed with one dedicated testbench per synthesizable module. The ten testbenches contain 114 VHDL assertion statements covering input capture, priority mapping, allocation and release, patient parking, state transitions, display encoding, alarm latching, LED mapping, UART framing,

TABLE I
FUNCTIONAL VERIFICATION SCOPE

Function	Verified behavior
Patient acquisition	Synchronized capture and single-event response
Priority logic	Severity/type combinations mapped correctly
Resource manager	Allocation, occupancy, exhaustion, and release
Scheduler FSM	Normal, parking, emergency, fault, and reset paths
Patient table	Multi-slot storage and selective release
Indicators	LED, alarm blink, and seven-segment outputs
UART	Start/data/stop framing and transmission sequencing
Top-level	Integrated interaction of controls and resources

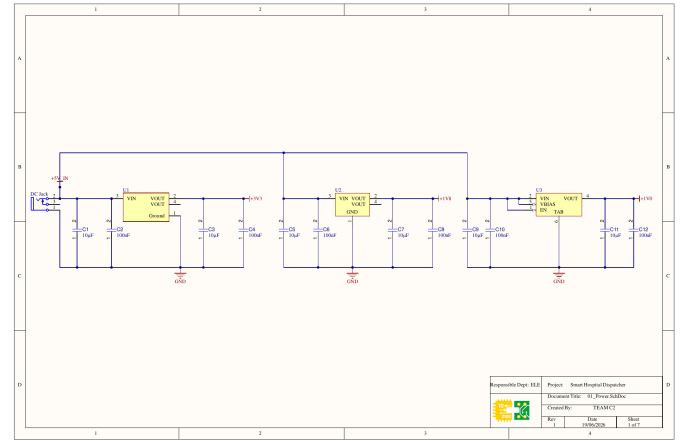


Fig. 3. Altium power-supply schematic used for the Smart Hospital Dispatcher interface board.

and top-level behavior. This modular strategy improves fault localization and avoids relying only on a final demonstration.

IV. ALTUM SCHEMATIC AND PCB REALIZATION

The custom interface-board design was developed in Altium Designer. The schematic set is hierarchically separated into power, input/control, display/output, FPGA interface, clock/configuration, and communication functions. This organization supports reviewability and traceability from logical function to PCB connectivity [11].

Figure 3 shows the power section. It provides the regulated supply structure and local decoupling needed by the digital interfaces. Power-domain separation and clear rail naming are important because the board combines logic, display, and peripheral loads.

The user-input interface in Fig. 4 connects slide switches and push buttons to the FPGA-facing signals. These inputs represent patient data, severity, resource-control operations, emergency override, release selection, and reset functions.

Figure 5 presents the display and indication circuitry. The implementation exposes system states through LEDs and seven-

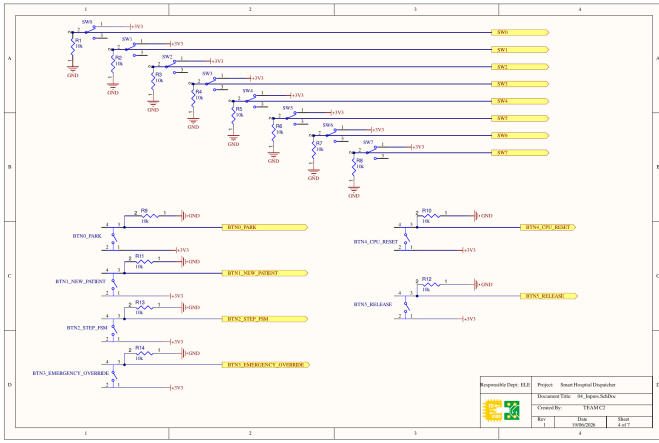


Fig. 4. Altium input-interface schematic for switches and push-button controls.

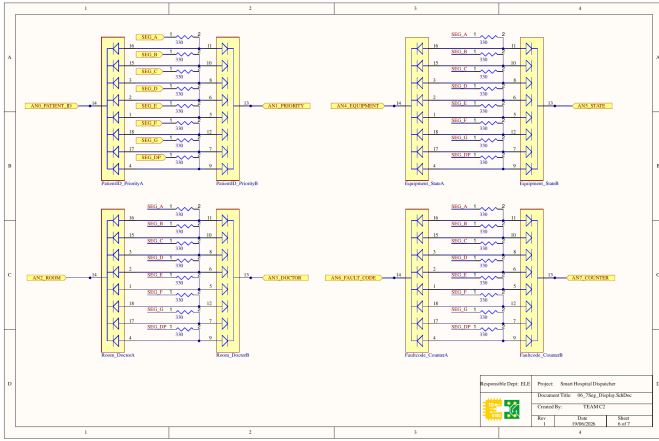


Fig. 5. Altium display-interface schematic for visual status and seven-segment outputs.

segment displays, allowing occupancy, fault, priority, and scheduler information to be observed during testing.

The routed two-layer PCB is shown in Fig. 6. Component placement keeps user controls and indicators accessible while routing the FPGA-interface signals toward the board connector. The design package includes top and bottom copper, solder mask, legend, paste, board profile, plated-through-hole drill, and non-plated-through-hole drill outputs.

Figure 7 provides the three-dimensional board review. The model enables inspection of connector orientation, control accessibility, display placement, board proportions, and potential mechanical conflicts before fabrication.

V. LABORATORY DEMONSTRATION AND FUNCTIONAL SCENARIOS

The generated bitstream was programmed on the Nexys A7 and evaluated interactively in the laboratory. Figure 8 is the team's final working hardware image and is the physical

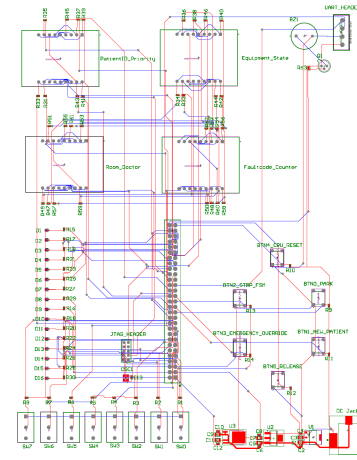


Fig. 6. Individually presented two-layer PCB layout exported from Altium Designer.

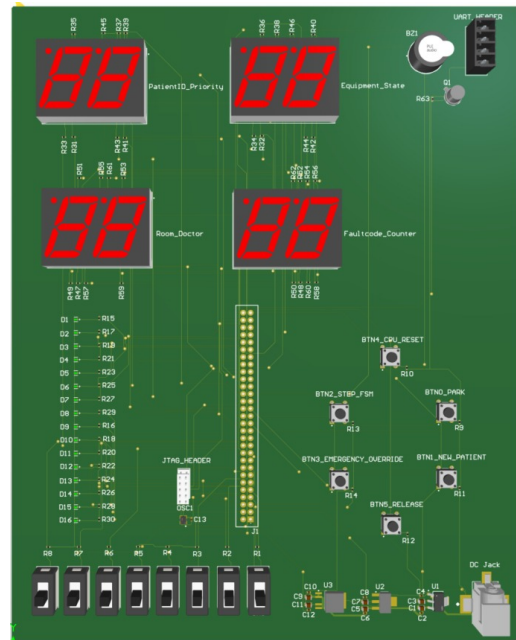


Fig. 7. Three-dimensional Altium rendering of the custom Smart Hospital Dispatcher PCB.

realization evidence used in this report. It replaces all non-team realization images.

The functional scenarios were prepared as individual visual records. They are not combined into a collage, allowing every sequence and label to remain readable.

A. Normal Patient Dispatch

Figure 9 documents the normal urgent-patient path. The system captures the patient, performs triage, checks resources, allocates a room, doctor, and equipment, dispatches the team, enters treatment, releases the resources, and returns to IDLE.



Fig. 8. Team C2 final working hardware demonstration on the Digilent Nexys A7 FPGA board.

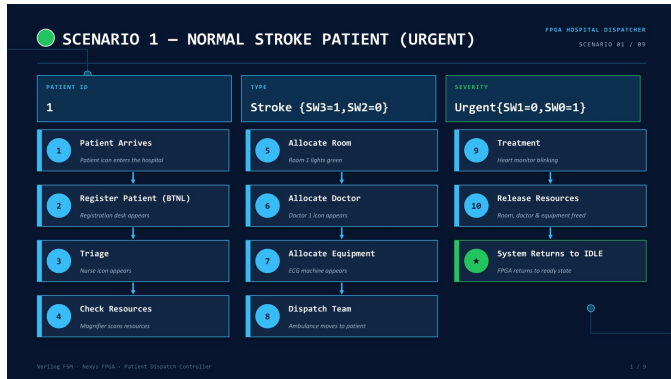


Fig. 9. Scenario 1: normal urgent-patient registration, allocation, treatment, release, and return to the idle state.

B. Critical Patient Handling

Figure 10 validates the critical-heart-attack path. The critical indication is asserted, the priority is raised to its highest level, resources are assigned, treatment duration is extended, and the flag is cleared after release.

C. Emergency Override

The emergency override sequence is shown in Fig. 11. A normal case can be escalated through the dedicated override control, forcing the critical path, activating the critical LED, and fast-tracking allocation and dispatch.

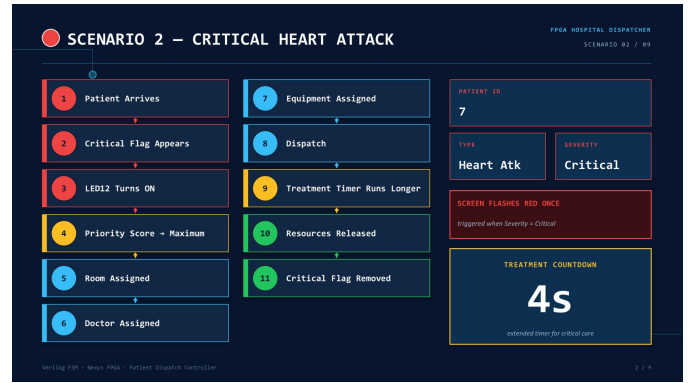


Fig. 10. Scenario 2: critical patient handling with maximum priority and extended treatment indication.



Fig. 11. Scenario 3: emergency override (BTNR) escalating a normal patient to the critical fast-track path.

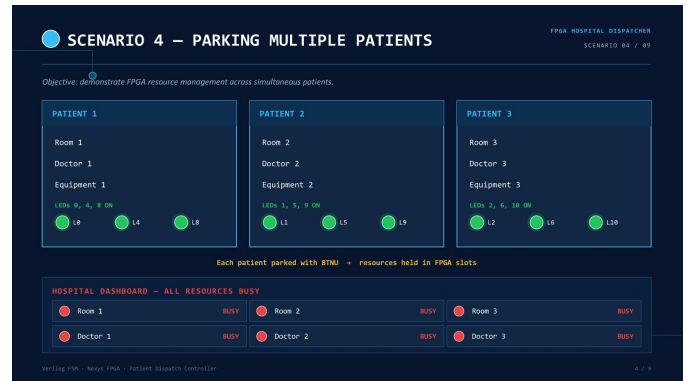


Fig. 12. Scenario 4: parking multiple patients while preserving their allocated resources in FPGA storage slots.

D. Multiple-Patient Parking

Figure 12 demonstrates retained allocations for multiple patients. Parking stores each assignment in the patient table and keeps the associated room, doctor, and equipment occupied until an explicit release is requested.

E. Selective Release

Figure 13 shows selective release. The operator chooses a stored slot, arms the release function, confirms it, and frees

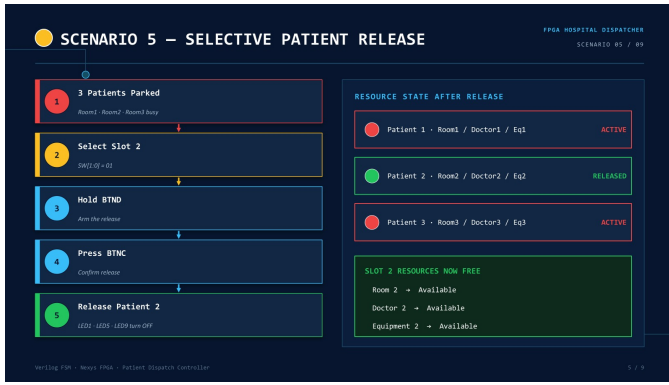


Fig. 13. Scenario 5: selective release of one parked patient without disturbing the remaining active allocations.

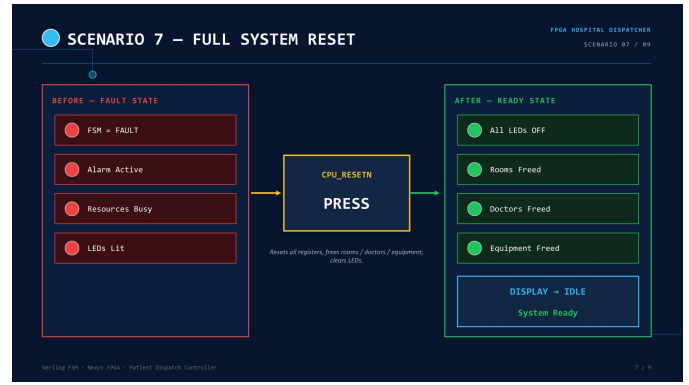


Fig. 15. Scenario 7: complete reset from the fault state to the ready/idle condition.

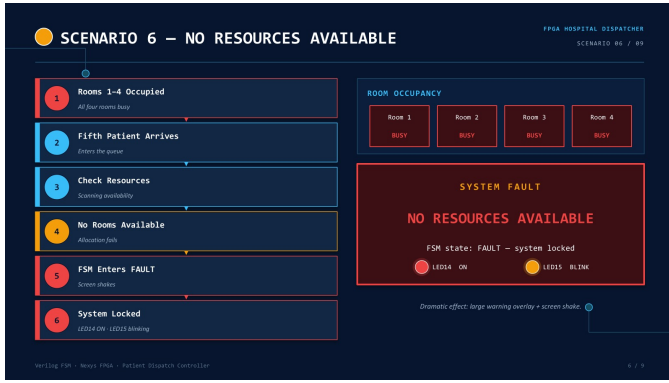


Fig. 14. Scenario 6: resource exhaustion causing controlled entry into the fault state and visible alarm indication.

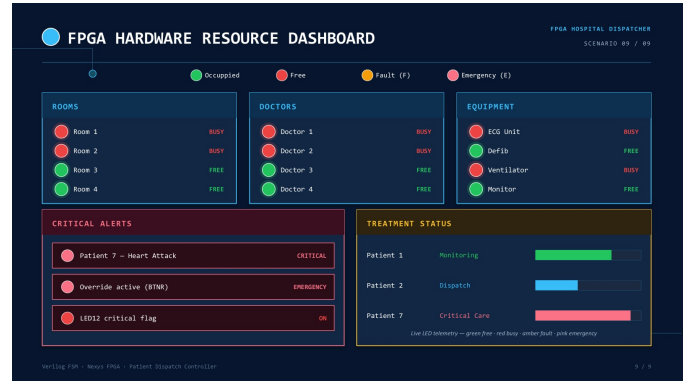


Fig. 16. FPGA hardware resource dashboard showing occupancy, availability, critical alerts, and treatment status.

only the room, doctor, and equipment belonging to the selected patient.

F. Resource Exhaustion

Figure 14 validates the no-resource condition. After all rooms are occupied, another admission cannot be allocated safely. The FSM enters the fault state, the system is locked, and the alarm indicators become active.

G. Full System Reset

Figure 15 documents recovery from the fault condition. Activating the reset clears state registers, occupancy, parked assignments, LEDs, and the latched alarm, restoring the ready state.

H. Hardware Resource Dashboard

Figure 16 summarizes the final occupancy and alert state. It displays room, doctor, and equipment status together with critical alerts and treatment information, providing an integrated view of the resource manager and scheduler outputs.

VI. RESULTS AND ENGINEERING ASSESSMENT

The project achieved its primary objectives. The RTL architecture is modular, each design unit has a corresponding testbench, the complete design was constrained and converted

into a board-programming bitstream, and the intended scenarios were demonstrated in the laboratory. The visual records show normal dispatch, critical handling, emergency override, multiple-patient parking, selective release, controlled resource-exhaustion behavior, reset recovery, and aggregate dashboard status.

The PCB work extends the project beyond an FPGA-only demonstration. The schematic sheets, routed board, three-dimensional model, bill of materials, Gerber set, and drill files form a manufacture-oriented design package. The assembled custom PCB was not part of the final laboratory evidence, so the report distinguishes the successfully demonstrated Nexys A7 implementation from the design-stage PCB realization.

Several improvements are recommended for a production-oriented revision. Mechanical inputs should use explicit debounce logic; incoming patients could be queued instead of operating on one active transaction; safety requirements could be formalized as temporal properties; UART messages could include framing and integrity checks; and the PCB should undergo documented electrical-rule checking, design-rule checking, regulator validation, connector pin review, and prototype bring-up. These items are refinements rather than changes to the fundamental architecture.

VII. CONCLUSION

Team C2 completed an end-to-end Smart Hospital Emergency Response Dispatcher integrating VHDL design, verification, FPGA deployment, laboratory demonstration, and Altium PCB engineering. The system comprises ten synthesizable modules and ten testbenches with 114 assertions. It supports triage, resource allocation, emergency escalation, patient parking, selective release, fault handling, visual indication, and serial reporting. The successful Nexys A7 demonstration confirms the interactive FPGA realization, while the schematics, PCB layout, three-dimensional model, bill of materials, and Gerber/drill outputs document the path toward dedicated hardware. Presenting every project figure individually provides clear and traceable evidence of the team's technical achievements.

ACKNOWLEDGMENT

The authors thank the Hardware Engineering Laboratory teaching staff at Hochschule Hamm-Lippstadt for the laboratory platform, technical guidance, and opportunity to validate the design on physical FPGA hardware.

REFERENCES

- [1] Diligent, *Nexys A7 FPGA Board Reference Manual*, 2024.
- [2] AMD, *Artix 7 FPGAs Data Sheet: DC and AC Switching Characteristics*, 1st ed., Jul. 2024, product specification.
- [3] IEEE, *IEEE Standard for VHDL Language Reference Manual*, 2019.
- [4] —, *IEEE Standard Multivalued Logic System for VHDL Model Interoperability*, 1993.
- [5] AMD, *Vivado Design Suite User Guide: Logic Simulation*, 2024.
- [6] —, *Vivado Design Suite User Guide: Synthesis*, 2025.
- [7] —, *Vivado Design Suite User Guide: Using Constraints*, 2024.
- [8] —, *Vivado Design Suite User Guide: Implementation*, 2024.
- [9] P. P. Chu, *FPGA Prototyping by VHDL Examples: Xilinx MicroBlaze MCS SoC*, 2nd ed. Wiley, 2017.
- [10] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. Pearson, 2018.
- [11] Altium, *Altium Designer Documentation: Schematic Capture, PCB Layout, and Manufacturing Outputs*, 2026.